IE221 - BÀI THỰC HÀNH 5
MỘT SỐ CHỦ ĐỀ MỞ RỘNG

Ngày 3 tháng 12 năm 2025

Mục lục

1. RÚT TRÍCH DỮ LIỆU TỪ WEB **2**
- 1.1 Bài tập 1: Crawl dữ liệu từ VNExpress . . . 2
- 1.2 Bài tập 2: Crawl nhiều trang từ VNExpress . . . 2
- 1.3 Bài tập 3: Crawl dữ liệu từ Báo Thanh Niên - Đa dạng chủ đề . . . 3

2. MỘT SỐ THAO TÁC VỚI DỮ LIỆU ĐA PHƯƠNG TIỆN **4**
- 2.1 Thao tác với dữ liệu văn bản . . . 4
  - 2.1.1 Bài tập 4: Phân tích tần suất từ khóa từ tiêu đề bài báo . . . 4
- 2.2 Thao tác với dữ liệu âm thanh . . . 4
  - 2.2.1 Bài tập 5: Tải, phát và chỉnh sửa file âm thanh . . . 4
  - 2.2.2 Bài tập 6: Gộp hai file âm thanh . . . 5
- 2.3 Thao tác với dữ liệu hình ảnh . . . 5
  - 2.3.1 Bài tập 7: Tải, đọc và hiển thị hình ảnh . . . 5
  - 2.3.2 Bài tập 8: Chuyển đổi không gian màu . . . 6
  - 2.3.3 Bài tập 9: Vẽ các hình vẽ và text trên ảnh . . . 7
- 2.4 Thao tác với dữ liệu video . . . 7
  - 2.4.1 Bài tập 10: Tải, phát và trích xuất thông tin video . . . 7
  - 2.4.2 Bài tập 12: Cắt video thành các đoạn nhỏ và tách âm thanh . . . 7

1. RÚT TRÍCH DỮ LIỆU TỪ WEB

1.1 Bài tập 1: Crawl dữ liệu từ VNExpress

Mục tiêu: Sử dụng thư viện Requests và Beautiful Soup để lấy dữ liệu từ trang báo điện tử VNExpress.

Yêu cầu:

1. Sử dụng thư viện Requests để gửi HTTP GET request đến trang chủ VNExpress (`https://vnexpress.net`) hoặc một mục tin cụ thể (ví dụ: `https://vnexpress.net/thoi-su`).
2. Phân tích cấu trúc HTML của trang báo VNExpress bằng cách sử dụng công cụ Developer Tools (F12 trong trình duyệt) để xác định:
   - Thẻ HTML chứa tiêu đề bài báo
   - Các thuộc tính class hoặc id của tiêu đề
   - Cấu trúc lồng nhau của các phần tử
3. Sử dụng Beautiful Soup để parse HTML response và trích xuất **tiêu đề của 20 bài báo** từ trang đầu tiên.
4. Tìm thẻ `<a>` (anchor tag) chứa URL và trích xuất giá trị thuộc tính `href`. Lưu kết quả vào một danh sách các dictionary có cấu trúc:

```
[
    {"title": "Tiêu đề bài 1", "url": "https://vnexpress.net/..."},
    {"title": "Tiêu đề bài 2", "url": "https://vnexpress.net/..."},
    ...
]
```

5. Tìm vị trí chứa text mô tả (thường là một thẻ `<p>` hoặc `<span>` sau tiêu đề) và trích xuất. Xử lý trường hợp một số bài báo có mô tả, một số thì không.
6. Lưu dữ liệu vào file JSON hoặc CSV.

Code gợi ý:

```python
import requests
from bs4 import BeautifulSoup

# Gửi GET request
url = "https://vnexpress.net/thoi-su-p1"
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
}

try:
    response = requests.get(url, headers=headers, timeout=5)
    response.raise_for_status()  # Kiểm tra HTTP status code

    # Parse HTML
    soup = BeautifulSoup(response.content, 'html.parser')

    # Tìm tiêu đề bài báo (cần kiểm tra cấu trúc HTML thực tế)
    articles = soup.find_all('article', class_='item-news')
```

2. Thêm độ trễ giữa các request bằng `time.sleep(2)` để tránh làm quá tải máy chủ.
3. Tổng hợp tất cả bài báo từ 5 trang vào một danh sách duy nhất.
4. Thống kê và in ra:
   - Tổng số bài báo crawl được
   - Số bài báo trung bình trên mỗi trang

1.3 Bài tập 3: Crawl dữ liệu từ Báo Thanh Niên - Đa dạng chủ đề

Mục tiêu: Chuyển sang crawl dữ liệu từ Báo Thanh Niên (https://thanhnien.vn), lấy đầy đủ thông tin một bài báo và crawl từ nhiều chuyên mục khác nhau.
Yêu cầu:

1. Phân tích cấu trúc HTML của Báo Thanh Niên để xác định:
   - Các chuyên mục chính: Thời sự, Kinh tế, Giáo dục, Thể thao, Văn hóa, ...
   - URL của từng chuyên mục (ví dụ: https://thanhnien.vn/thoi-su.htm)
   - Cấu trúc HTML chứa bài báo trên trang danh sách
2. Crawl **đầy đủ thông tin** của mỗi bài báo bao gồm:
   - Tiêu đề (title)
   - URL bài báo
   - Mô tả/Tóm tắt (description/summary)
   - Tác giả (author) - nếu có
   - Ngày xuất bản (publish_date) - nếu có
   - Chuyên mục/Danh mục (category)
   - Hình ảnh đại diện (thumbnail_url) - nếu có
3. Crawl dữ liệu từ **ít nhất 3 chuyên mục khác nhau**, mỗi chuyên mục lấy 10 bài báo. Các chuyên mục gợi ý:
   - Thời sự: https://thanhnien.vn/thoi-su.htm
   - Kinh tế: https://thanhnien.vn/kinh-te.htm
   - Giáo dục: https://thanhnien.vn/giao-duc.htm
   - Công nghệ: https://thanhnien.vn/cong-nghe.htm
   - Thể thao: https://thanhnien.vn/the-thao.htm
4. Đối với mỗi bài báo, truy cập vào **trang chi tiết** để lấy thêm thông tin:
   - Nội dung đầy đủ của bài báo (full_content)
   - Số lượt xem (views) - nếu có
   - Thẻ tag (tags) - nếu có
5. Xử lý lỗi và các trường hợp đặc biệt:
   - Một số trường thông tin có thể không tồn tại
   - Xử lý timeout khi crawl
   - Thêm delay giữa các request để tránh bị chặn
6. Lưu dữ liệu vào file JSON với cấu trúc phân cấp theo chuyên mục:

```json
{
    "thoi_su": [
        {
            "title": "...",
            "url": "...",
            "description": "...",
            "author": "...",
            "publish_date": "...",
            "category": "thoi_su",
```

```
            "thumbnail_url": "...",
            "full_content": "...",
            "tags": [...]
        },
        ...
    ],
    "kinh_te": [...],
    "giao_duc": [...]
}
```

7. Thống kê và in ra:
   - Tổng số bài báo crawl được từ mỗi chuyên mục
   - Tổng số bài báo crawl được
   - Số bài báo có đầy đủ thông tin (author, publish_date)
   - Số bài báo thiếu thông tin

2. MỘT SỐ THAO TÁC VỚI DỮ LIỆU ĐA PHƯƠNG TIỆN

2.1 Thao tác với dữ liệu văn bản

2.1.1 Bài tập 4: Phân tích tần suất từ khóa từ tiêu đề bài báo

Mục tiêu: Sử dụng dữ liệu đã crawl từ VNExpress hoặc Thanh Niên để thực hiện xử lý văn bản.
Yêu cầu:

1. Lấy **tiêu đề của 50 bài báo** từ VNExpress hoặc Thanh Niên (hoặc từ file JSON đã lưu ở trước).
2. Thực hiện các bước xử lý văn bản:
   - Chuyển tất cả ký tự thường (`.lower()`)
   - Loại bỏ dấu câu (sử dụng `string.punctuation` hoặc regex)
   - Tách từ dựa trên dấu cách (`.split()`)
3. Thống kê tần suất xuất hiện của từng từ khóa.
4. Sắp xếp theo tần suất giảm dần và lấy **top 15 từ khóa** có tần suất cao nhất.
5. Vẽ biểu đồ cột (bar chart) biểu diễn tần suất của 15 từ khóa này.

2.2 Thao tác với dữ liệu âm thanh

2.2.1 Bài tập 5: Tải, phát và chỉnh sửa file âm thanh

Chuẩn bị: Tải một file âm thanh (ví dụ: `background_music.wav` hoặc `speech.mp3`)
Yêu cầu:

1. Sử dụng thư viện **PyDub** để:
   - Tải file âm thanh từ đĩa cứng
   - Phát file âm thanh
   - Lấy thông tin của file (thời lượng, sample rate, số kênh)
2. Sử dụng thư viện **wave** hoặc **soundfile** để:
   - Đọc dữ liệu âm thanh từ file WAV
   - Lấy các thông số: sample rate, số sample, độ sâu bit
3. Thực hiện các thao tác xử lý:
   - Trích xuất 5 giây đầu tiên của file âm thanh
   - Lấy 10 giây giữa file (từ giây thứ 15 đến giây thứ 25)
   - Thay đổi âm lượng (tăng hoặc giảm dB)
   - Lưu file âm thanh được xử lý thành file WAV mới
4. Lặp lại file âm thanh 3 lần (overlap 3 bản)

2.2.2 Bài tập 6: Gộp hai file âm thanh

Yêu cầu:

1. Tải hai file âm thanh khác nhau (ví dụ: `song1.wav` và `song2.wav`).
2. Gộp hai file theo thứ tự: song1 + song2.
3. Gộp hai file với khoảng trống 2 giây ở giữa:
   - Tạo khoảng trống im lặng 2 giây
   - song1 + khoảng trống + song2
4. Gộp hai file với chồng lặp 1 giây cuối của song1 với 1 giây đầu của song2.
5. Lưu cả 3 kết quả thành các file WAV khác nhau.

Code gợi ý:

```python
from pydub import AudioSegment

# Tải hai file âm thanh
song1 = AudioSegment.from_file("song1.wav", format="wav")
...

# Gộp trực tiếp
merged_direct = song1 + song2
merged_direct.export("merged_direct.wav", format="wav")
print(f"\n1. Gộp trực tiếp: {len(merged_direct) / 1000:.2f} giây")

# Gộp với khoảng trống 2 giây
silence = AudioSegment.silent(duration=2000)  # 2000ms = 2 giây
merged_with_silence = ...

# Gộp với chồng lặp 1 giây
overlap_duration = 1000  # 1 giây
song1_before_overlap = song1[:-overlap_duration]
merged_overlapped = ...
...
```

2.3 Thao tác với dữ liệu hình ảnh

2.3.1 Bài tập 7: Tải, đọc và hiển thị hình ảnh

Chuẩn bị: Tải một file ảnh (ví dụ: `photo.jpg` hoặc `image.png`)
Yêu cầu:

1. Sử dụng thư viện **OpenCV (cv2)** để:
   - Tải ảnh màu từ file
   - Tải ảnh xám (grayscale)
   - Hiển thị cả hai ảnh trong các cửa sổ riêng biệt
2. Lấy thông tin ảnh:
   - Kích thước ảnh (width × height)
   - Số chiều của numpy array
   - Số kênh màu (RGB: 3 kênh, Grayscale: 1 kênh)
3. Thay đổi kích thước ảnh thành:
   - 300 × 400 pixel
   - 800 × 600 pixel
   - Lưu các ảnh đã resize
4. Cắt (crop) ảnh:
   - Cắt vùng từ tọa độ $(50, 50)$ với kích thước 200 × 200

- Cắt vùng từ tọa độ $(100, 100)$ với kích thước 150×150
- Lưu các ảnh đã cắt

Code gợi ý:

```python
import cv2
import numpy as np

# Tải ảnh màu (BGR format trong OpenCV)
image_color = cv2.imread('photo.jpg')

# Tải ảnh xám
image_gray = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)

# Lấy thông tin ảnh
print(f"Ảnh màu shape: {image_color.shape}")
...

# Hiển thị ảnh
cv2.imshow('Color Image', image_color)
...

# Thay đổi kích thước (resize)
resized_300x400 = cv2.resize(image_color, (300, 400))
...

cv2.imwrite('resized_300x400.jpg', resized_300x400)
...

# Cắt ảnh (crop)
# Cú pháp: image[y1:y2, x1:x2]
cropped_200x200 = image_color[50:250, 50:250]   # Từ (50,50) kích thước 200x200

...
```

2.3.2 Bài tập 8: Chuyển đổi không gian màu

Yêu cầu:

1. Tải ảnh gốc (ảnh màu BGR).
2. Chuyển đổi ảnh sang các không gian màu khác nhau:
   - RGB (từ BGR)
   - Grayscale
   - HSV (Hue-Saturation-Value)
   - LAB
   - YUV
3. Hiển thị từng ảnh đã chuyển đổi trong cửa sổ riêng hoặc subplot.
4. Lưu tất cả các ảnh đã chuyển đổi thành file ảnh khác nhau (với đặc điểm riêng biệt).

Code gợi ý:

```python
import cv2
import matplotlib.pyplot as plt

# Tải ảnh gốc
image = cv2.imread('photo.jpg')

# Chuyển đổi không gian màu
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
...

# Vẽ các ảnh trong một subplot

...

print("So sánh không gian màu đã được lưu vào: color_space_comparison.png")
```

2.3.3 Bài tập 9: Vẽ các hình vẽ và text trên ảnh

Yêu cầu:

1. Tạo một ảnh trắng (hoặc tải ảnh gốc).
2. Vẽ các hình dạng khác nhau:
   - Đường thẳng: từ điểm $(0, 0)$ đến điểm $(400, 300)$ - màu xanh, độ dày 3
   - Hình chữ nhật: tọa độ $(50, 50)$ đến $(250, 200)$ - màu đỏ, độ dày 2
   - Hình tròn: tâm $(300, 150)$, bán kính 80 - màu xanh lá, độ dày 2
   - Hình ellipse: tâm $(200, 250)$, kích thước $(100, 50)$ - màu vàng, độ dày 2
   - Đa giác (polygon): các đỉnh $[(50, 100), (150, 50), (200, 150)]$ - màu tím, độ dày 2
3. Viết text trên ảnh:
   - "Python Programming" tại vị trí $(50, 50)$ - màu trắng, kích thước 1.5
   - "Image Processing" tại vị trí $(50, 100)$ - màu đen, kích thước 1.0
4. Lưu ảnh đã vẽ.

2.4 Thao tác với dữ liệu video

2.4.1 Bài tập 10: Tải, phát và trích xuất thông tin video

Chuẩn bị: Tải một file video (ví dụ: `sample_video.mp4`)
Yêu cầu:

1. Sử dụng thư viện **OpenCV (cv2.VideoCapture)** để:
   - Tải và mở file video
   - Lấy thông tin video: độ phân giải (width, height), FPS, tổng số frame
   - Phát video toàn bộ (hiển thị từng frame)
2. Đếm và thống kê tổng số frame trong video.
3. Lưu thông tin video vào file text.
4. Trích xuất 5 frame đầu tiên từ video và lưu thành file ảnh (JPG).

Code gợi ý:

2.4.2 Bài tập 12: Cắt video thành các đoạn nhỏ và tách âm thanh

Yêu cầu:

1. Sử dụng thư viện **MoviePy** để:
   - Tải file video
   - Cắt video thành các đoạn 10 giây
   - Lưu mỗi đoạn thành file video riêng biệt
2. Tách âm thanh từ video:
   - Trích xuất âm thanh từ video
   - Lưu âm thanh thành file WAV hoặc MP3
3. Lấy thông tin về video:
   - Độ phân giải
   - Thời lượng
   - Frame rate

Nộp bài: Chạy code bài tập trên Colab hoặc các IDE (.py), thực hiện lại demo và làm các bài tập về nhà theo yêu cầu. Nếu thực hiện trên các IDE thì nộp kèm các file code kèm theo 1 pdf kết quả chạy. Đặt tên MSSV_HoVaTen_Lab6.pdf và nộp theo deadline trên Courses.